

Data Science Life Cycle

Data Collection: Gathering raw data.

Data Processing: Cleaning and organizing data.

Data Analysis: Applying statistical/machine learning models.

Interpretation: Extracting insights and conclusions.

Key Definitions

AI: Creating machines that think and act like humans.

Machine Learning: Mapping function from input to output through training.

Data Science: Identifying patterns to make predictions.

Python: High-level, interpreted language with dynamic semantics.

Pandas: Data analysis library for fast and convenient data manipulation. Speeds up data cleaning, exploration, and analysis. Provides easy tools for data alignment, reshaping, and handling missing data.

Modelling: Creating simplified real-world system representations.

Pandas Essentials

Series: 1D labeled array (like a column).

DataFrame: 2D tabular data with rows/columns.

Index: The labels for rows and columns, making data manipulation easier by accessing data through labels instead of positions.

`elections = pd.read_csv("elections.csv")`

Data Slicing Techniques

Basic: `Df['column_name']`

#Access a single column

`df[['col1', 'col2']]`

#Access multiple columns

Boolean Indexing:

`df[df['Year'] == 1828]['Candidate']`

#Filter rows where Year = 1828, then show Candidate

Loc (Label-based):

`df.loc[row_condition, column_selection]`

`df.loc[df['Party'] == 'Democratic']` #Filter rows where Party = Democratic

`df.loc[df['Result'] == 'win', ['Year', 'Popular vote']]`

#Filter & show specific columns

Iloc (Index-based): `df.iloc[0:5, 2:4]`

#Select rows 0-4 and columns 2-3 (by index)

Grouping and Aggregation

Groupby:

`df.groupby('column_name').agg_function()`

#e.g., `.mean()`, `.sum()`, `.count()`

Pivot Tables:

`df.pivot_table(values='value', index='index_col', columns='col', aggfunc='mean')`

Simple Linear Regression:

Equation: $y = mx + b$

- y (Dependent Variable):** The outcome you want to predict.
- x (Independent Variable):** The predictor or input variable.
- m:** Slope (rate of change).
- b:** Intercept (value of y when x = 0).

Steps: Calculate the slope (m) and intercept (b) using data. Substitute x values into the equation to predict y.

Significance:

- Identifies and quantifies relationships between two variables.

- Used in various fields like finance, biology, and economics for trend analysis and forecasting.

What: Models the relationship between two variables with a straight line.

Why: To predict one variable based on another.

When: Used for trend analysis and forecasting.

Error Metrics:

Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- Measures average squared differences between actual and predicted values.
- Sensitive to outliers** due to squaring errors.

Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- Measures average absolute differences between actual and predicted values.
- Less sensitive to outliers** compared to MSE.

Numpy Quick Reference

Array Creation: `import numpy as np`

`arr = np.array([1, 2, 3])`

Common Functions: `np.mean(arr)`, `np.sum(arr)`, `np.max(arr)`, `np.min(arr)`

Practice Problems

Access 'Candidates' where Year = 1828:

`df[df['Year'] == 1828]['Candidate']`

Filter rows where Party is 'Democratic':

`df.loc[df['Party'] == 'Democratic']`

Display Year & Popular vote where Result is 'win':

`df.loc[df['Result'] == 'win', ['Year', 'Popular vote']]`

Data Science Life Cycle

What: A process to extract meaningful insights from raw data. **Why:** Ensures a structured approach to solving real-world problems with data. **When:** Applied in any data-driven decision-making process. **Where:** Used across industries (e.g., healthcare, finance, marketing).

Artificial Intelligence (AI)

Why: To automate tasks, improve efficiency, and solve complex problems. **How:** Involves programming machines to learn, reason, and act. **When:** When tasks require decision-making or pattern recognition. **Who:** Researchers, developers, data scientists. **Where:** From robotics to chatbots to self-driving cars.

Machine Learning (ML)

Why: To create predictive models without explicit programming. **How:** Algorithms train on data to find patterns. **When:** Used when rules are difficult to define manually. **Where:** Applications in recommendation systems, fraud detection, etc.

Python

Why: Easy syntax, wide libraries, versatile for data science.

How: Code written in Python is executed line-by-line.

When: Ideal for rapid development and prototyping.

Where: Web development, data analysis, AI, automation.

Pandas

What: A Python library for data manipulation and analysis.

Why: Simplifies data handling with structures like DataFrames and Series. **How:** Provides functions to read, clean, and analyze data. **When:** When working with tabular data in Python. **Where:** Used in data science, finance, and any domain needing data analysis.

Series

What: A one-dimensional labeled array in pandas. **Why:** Represents a single column of data.

Index: Default (0, 1, ...) or custom.

Supports lists, dictionaries, arrays.

How: Accessed using index labels or position.

DataFrame

What: A two-dimensional labeled data structure in pandas.

Why: Allows handling of tabular data efficiently.

How: Created from lists, dictionaries, or reading files.

Data Slicing Techniques

Iloc

What: Label-based selection.

Why: Access data by row/column labels.

How: `df.loc[row_label, col_label]`

Iloc

What: Index-based selection.

Why: Access data by numerical index.

How: `df.iloc[row_index, col_index]`

Grouping and Pivot Tables

groupby

What: Groups data based on specific columns.

Why: For aggregate calculations like sum, mean.

How: `df.groupby('column').agg_function()`

Pivot Table

What: Reshapes data, aggregating based on values.

Why: Simplifies complex data summaries.

How: `df.pivot_table(values, index, columns)`

Creating a Pandas Series

From a List:

`series = pd.Series([10, 20, 30])`

#Default index: 0, 1, 2

From a Dictionary:

`series = pd.Series({'a': 100, 'b': 200})`

#Keys become the index

From a Numpy Array:

`import numpy as np`
`series = pd.Series(np.array([1.5, 2.5]),`

`index=['x', 'y'])` #Custom index

Lists vs. NumPy Arrays

Data Type:

Lists can store mixed data types (e.g., integers, strings, etc.).

NumPy arrays store homogeneous data, meaning all elements must be of the same type.

Performance:

Lists are slower due to dynamic typing and flexibility.

NumPy arrays are faster because they are optimized with C under the hood for numerical computations.

Memory Usage:

Lists consume more memory due to their dynamic nature and ability to store different data types.

NumPy arrays are more memory-efficient as they are specifically designed for numerical data.

Functionality:

Lists support basic operations like `append` and `remove`.

NumPy arrays support advanced operations, including mathematical functions, vectorized operations, and array manipulation.

Indexing:

Lists support simple indexing (accessing elements by position).

NumPy arrays allow advanced slicing, including multi-dimensional slicing and broadcasting.

Libraries:

Lists are built into Python and don't require any external libraries.

NumPy arrays require importing the `numpy` library (`import numpy as np`).

2. Using iloc

`iloc[]` is used for integer-location based indexing (by row/column positions).

`df.iloc[0, 1]` # First row, second column

`df.iloc[0:3, 1:4]` # Rows 0-2, columns 1-3

3. Using loc

`loc[]` is used for label-based indexing (by row/column names).

`df.loc['row_label', 'col_label']` # Specific row and column by labels

`df.loc[0:2, 'col1':'col3']` # Rows 0-2, columns from 'col1' to 'col3'

4. Bracket [] Indexing

Brackets can be used to select columns or filter data.

`df['col_name']` # Select a column

`df[df['col_name'] > value]` # Filter rows where a condition is met

5. Slicing: Used to get a range of rows or columns.

`df[0:3]` # First 3 rows

`df.loc[:, 'col1':'col3']` # All rows, columns 'col1' to 'col3'

How many rows and columns?

`df.shape`

2. Selecting Data with []:

Display the list of candidates: `df['Candidate']`

Display all data for the election year 1828:

`df[df['Year'] == 1828]`

Selecting Data with loc:

Display all rows where the candidate's party is "Democratic":

`df.loc[df['Party'] == 'Democratic']`

Select columns 'Year' and 'Popular vote' for elections where the result is 'win':

`df.loc[df['Result'] == 'win', ['Year', 'Popular vote']]`

Find all elections where the percentage of the popular vote was above 55%: `df.loc[df['%'] > 55]`

Selecting Data with iloc:

Display the first 5 rows: `df.iloc[:5]`

Select data from the second to the fourth row and the first three columns: `df.iloc[1:4, :3]`

Extract 'Popular vote' and 'Result' columns for the first 10 rows: `df.iloc[:10, [3, 4]]`

Combining Conditions with loc and []:

Display all rows where the result was 'loss' and the percentage of votes was less than 50%:

`df.loc[(df['Result'] == 'loss') & (df['%'] < 50)]`

Display the candidates from the

'Democratic-Republican' party who won an election:

`df.loc[(df['Party'] == 'Democratic-Republican') & (df['Result'] == 'win'), 'Candidate']`

Bonus Challenge:

Create a new column 'Vote Margin' and display the top 5 candidates with the largest vote margin:

`df['Vote Margin'] = df['Popular vote'] -`

`df.groupby('Year')['Popular vote'].transform('mean')`

`df.nlargest(5, 'Vote Margin')[['Candidate', 'Vote Margin']]`

Boolean Arrays:

You create a Boolean array by applying a condition to a NumPy array or Pandas DataFrame.

`import numpy as np`

`arr = np.array([1, 2, 3, 4, 5])`

`bool_arr = arr > 3` # [False, False, False, True, True]

Using Boolean Arrays for Filtering:

Use a Boolean array to select elements that meet a condition.

`result = arr[bool_arr]` # [4, 5]

Boolean Arrays in Pandas:

You can filter rows in a DataFrame based on conditions.

`import pandas as pd`

`df = pd.DataFrame({'A': [10, 20, 30], 'B': [40, 50, 60]})`

`filtered_df = df[df['A'] > 15]` # Rows where column 'A' > 15

Combining Multiple Conditions:

Use logical operators like `&` (AND) or `|` (OR).

`filtered_df = df[(df['A'] > 15) & (df['B'] < 55)]` # Multiple conditions

Ensure that the Boolean array has the same length as the array or DataFrame being filtered.

Use parentheses `()` when combining multiple conditions to avoid errors.

Q: Which of the following pandas statements returns a DataFrame of the first 3 baby names with Count >

250: `babynames.loc[babynames["Count"] > 250, ["Name", "Count"]].iloc[0:2, :]`

`.isin()`

The `.isin()` method is used to filter rows based on whether the values in a column exist in a specified list or other iterable (like a set or another Series).

`df[column].isin(values)`

Filter rows where Name is either 'Alice' or 'Bob'

`df[df['Name'].isin(['Alice', 'Bob'])]`

`.str.startswith()`

The `.str.startswith()` method is used to check if each string in a Series starts with a given substring.

`df[column].str.startswith(substring)`

Filter rows where the Name starts with 'A'

`df[df['Name'].str.startswith('A')]`

`.groupby().filter()`

The `.groupby().filter()` method is used to filter data based on the group-level condition. It works by first applying a grouping operation and then filtering out entire groups based on a condition applied to each group.

`df.groupby('column').filter(lambda x: condition)`

column: The column to group by.

condition: A lambda function (or a condition function) that will be applied to each group. The group will be kept if the condition is **True**.

Keep only groups where the sum of 'Value' is greater than 50

```
df.groupby('Category').filter(lambda x: x['Value'].sum() > 50)
```

Custom Sorts

You can sort a DataFrame based on specific criteria using **sort_values**. Custom sorts allow you to control the order using either an external mapping or the logic inside your code.

Example: Sorting by a specific column

```
elections_sorted_by_percent = elections.sort_values("%", ascending=False)
```

This sorts the DataFrame in descending order based on the % column.

2. Adding, Modifying, and Removing Columns

You can add new columns, update existing ones, or delete them.

Add or Modify a Column:

```
elections['Vote Margin'] = elections['Popular vote'] - elections['Electoral vote']
```

Remove a Column:

```
elections = elections.drop(columns=['Vote Margin'])
```

1. Custom Sorts

You can sort a DataFrame based on specific criteria using **sort_values**. Custom sorts allow you to control the order using either an external mapping or the logic inside your code.

Example: Sorting by a specific column

```
elections_sorted_by_percent = elections.sort_values("%", ascending=False)
```

This sorts the DataFrame in descending order based on the % column.

2. Adding, Modifying, and Removing Columns

You can add new columns, update existing ones, or delete them.

Add or Modify a Column:

```
elections['Vote Margin'] = elections['Popular vote'] - elections['Electoral vote']
```

Remove a Column:

```
elections = elections.drop(columns=['Vote Margin'])
```

3. groupby().agg

The **.agg()** function is used to perform aggregate computations on groups. You can pass functions like **sum**, **mean**, **max**, or even custom lambda functions.

Example: Group by Party and Get the Maximum Percentage

```
elections.groupby("Party").agg(lambda x: x.iloc[0]) # First row of each group
```

4. Sorting by String Length

You can sort rows by the length of a column's string values.

Example:

```
elections['Name Length'] = elections['Candidate'].str.len() elections_sorted = elections.sort_values('Name Length')
```

5. groupby().size

The **.size()** method counts the number of rows in each group.

Example:

```
elections.groupby("Party").size()
```

This will return the number of elections for each party.

6. groupby().filter

Filters groups based on a condition applied to each group.

Example: Filter Parties with More Than 2 Elections

```
elections_filtered = elections.groupby("Party").filter(lambda x: len(x) > 2)
```

Pivot Tables

Pivot tables summarize data by reshaping it. Use **pivot_table()** to create them.

Example: Pivot with Party and Year

```
elections.pivot_table(values='%', index='Party', columns='Year', aggfunc='mean')
```

Using a Lambda Function

A lambda function is an anonymous function useful for custom computations.

Example: Sort and Group to Find the First Row

```
elections_sorted_by_percent = elections.sort_values("%", ascending=False) elections_sorted_by_percent.groupby("Party").agg(lambda x: x.iloc[0])
```

Using idxmax

The **idxmax** function finds the index of the maximum value in a Series.

Example: Find the Best Percentage for Each Party

```
best_per_party = elections.loc[elections.groupby("Party")["%"].idxmax()]
```

Using drop_duplicates

This method removes duplicate rows based on specified columns.

Example: Find the Best Percentage per Party Using drop_duplicates

```
best_per_party2 = elections.sort_values("%").drop_duplicates(["Party"], keep="last")
```

This sorts the DataFrame by %, removes duplicate rows based on **Party**, and keeps the last row (highest percentage for each party).

Summary

These methods are powerful for data manipulation in Pandas:

- **Custom sorts:** Organize data by any criteria.
- **Adding, modifying, removing columns:** Reshape your DataFrame as needed.

- **Groupby operations:** Summarize and filter groups effectively.
- **Pivot tables:** Reorganize data for aggregated views.
- **Lambda functions:** Apply custom logic.
- **idxmax and drop_duplicates:** Efficiently find key values.

Pandas and NumPy Basics:

1. Difference Between .loc and .iloc:

- **.loc:** Label-based indexing (uses row/column names).
- **.iloc:** Integer-based indexing (uses row/column positions).

2. What is the pandas library?

- A Python library for data manipulation and analysis with structures like DataFrames and Series.

3. Significance of pandas:

- Simplifies handling, cleaning, and analyzing structured data efficiently.

4. What is the NumPy library?

- A library for numerical computing, supporting arrays, matrices, and mathematical functions.

5. Significance of NumPy:

- Enables fast array operations and mathematical computations. It's the foundation for many data science libraries like pandas and scikit-learn.

Loss Functions:

6. Why is the Mean Squared Loss function used?

- Penalizes larger errors more heavily, making it sensitive to outliers; ideal for regression tasks.

7. Why is the Mean Absolute Loss function used?

- Treats all errors equally and is more robust to outliers compared to Mean Squared Loss.

Grouping in Pandas:

8. Why is grouping used in pandas?

- Aggregates data (e.g., sum, mean) based on unique values in a column, simplifying subset analysis.

Machine Learning:

9. What is a model in machine learning?

- A mathematical representation that learns patterns from data to make predictions or decisions.

10. Why are models important?

- They generalize patterns from data to solve real-world problems, such as regression or classification.

Key Questions and Answers:

11. What is the significance of pandas in Python?

- It provides tools for efficient data manipulation and analysis of structured data.

12. What is a DataFrame in pandas?

- A 2-dimensional, tabular data structure with labeled rows and columns.

13. What are the key data structures in pandas?

- **Series:** 1D data structure.
- **DataFrame:** 2D data structure.

More on .loc and .iloc:

14. How is .iloc different from .loc?

- **.iloc:** Integer-based indexing.
- **.loc:** Label-based indexing.

15. What does df.iloc[:,2] return?

- Every other row of the DataFrame, starting from the first.

Grouping and Aggregation:

16. What does df.groupby('col') do?

- Groups the DataFrame by unique values in the specified column.

17. How do you apply aggregation after grouping?

- Use **.agg()** or functions like **.sum()**, **.mean()** on the grouped object.

18. What does df.groupby('col').size() return?

- The number of rows in each group.

19. What does df.groupby('col').sum() return?

- The sum of values in each group for numerical columns.

20. Can you group by multiple columns?

- Yes, use **df.groupby(['col1', 'col2'])**.

Reading and Writing Data:

21. How do you read a CSV file in pandas?

- Use **pd.read_csv('file.csv')**.

22. How do you write a DataFrame to a CSV file?

- Use **df.to_csv('file.csv')**.

Handling Missing Values:

23. How do you check for missing values in a DataFrame?

- Use **df.isnull()** or **df.isnull().sum()**.

24. How do you drop rows with missing values?

- Use **df.dropna()**.

25. How do you replace missing values?

- Use **df.fillna(value)**.

Data Sorting:

26. How do you sort a DataFrame by a column?

- Use **df.sort_values('column')**.

Simple Linear Regression (SLR):

27. What is Simple Linear Regression?

- A method to model the relationship between one independent variable and one dependent variable.

28. What is the goal of Simple Linear Regression?

- To minimize the error between predicted and actual values.

29. What is Mean Squared Error (MSE)?

- The average squared difference between actual and predicted values.

30. What is Mean Absolute Error (MAE)?

- The average absolute difference between actual and predicted values.

31. How do you fit a linear regression model in Python?

Use libraries like **scikit-learn**:

python

Copy code

```
from sklearn.linear_model import LinearRegression model = LinearRegression() model.fit(X_train, y_train)
```

Three Reasons for Building Models:

1. Explain Complex Phenomena:

- Understand relationships between variables.
- Example: How do parents' heights relate to children's heights?
- Focus: Simple, interpretable models.

2. Make Accurate Predictions:

- Predict outcomes for unseen data.
- Example: Is an email spam?
- Focus: Accuracy over interpretability (e.g., black-box models in deep learning).

3. Infer Causation:

- Determine if one factor causes another.
- Example: Does smoking cause lung cancer?
- Focus: Advanced statistical methods (future courses).

Balancing **interpretability** and **accuracy** is key in most cases.

Q3 (a) [5 Pts] Suppose we have a dataset of one feature. We collect n observations of the form (x_i, y_i) . In other words, we record one scalar feature x_i and one scalar target variable y_i for the i -th datapoint. We model each observed y_i using a model with one scalar parameter, θ .

$$\hat{y}_i = f_{\theta}(x_i) = \theta \cdot \ln(|x_i| + 1)$$

Determine the optimal value $\hat{\theta}$ that minimizes the mean squared error (MSE) of this model on the dataset in terms of x_i, y_i . You may assume that the critical point of this loss function is guaranteed to be a minimum.

Step 1: Simplify the derivative

$$\frac{\partial L(\theta)}{\partial \theta} = \frac{1}{n} \sum_{i=1}^n 2(y_i - \theta \ln(|x_i| + 1)) \cdot (-\ln(|x_i| + 1))$$

Simplifying further:

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^n (y_i - \theta \ln(|x_i| + 1)) \ln(|x_i| + 1)$$

Step 2: Set the derivative to 0

To find the minimum of $L(\theta)$, we set the derivative to 0:

$$0 = -\frac{2}{n} \sum_{i=1}^n (y_i - \theta \ln(|x_i| + 1)) \ln(|x_i| + 1)$$

Canceling the $-2/n$ term:

$$0 = \sum_{i=1}^n (y_i - \theta \ln(|x_i| + 1)) \ln(|x_i| + 1)$$

Step 3: Solve for θ

Expand the summation:

$$0 = \sum_{i=1}^n y_i \ln(|x_i| + 1) - \theta \sum_{i=1}^n \ln(|x_i| + 1)^2$$

Rearrange:

$$\sum_{i=1}^n y_i \ln(|x_i| + 1) = \theta \sum_{i=1}^n \ln(|x_i| + 1)^2$$

Solve for θ :

$$\hat{\theta} = \frac{\sum_{i=1}^n y_i \ln(|x_i| + 1)}{\sum_{i=1}^n \ln(|x_i| + 1)^2}$$

Why Not Use Residual Error Directly?

Positive and negative residuals can cancel each other out, masking large errors and giving a false impression of accuracy.

When to Use MSE or MAE

MSE:

Key: Penalizes large errors (squares them).

Use for: Regression tasks, optimizing with gradient descent.

MAE:

Key: Treats all errors equally.

Use for: Robustness to outliers, minimizing average error.

Choosing the Metric:

Outliers? Use MAE.

Optimization? Use MSE.

Interpretability? MAE is easier to understand.

Summary:

MSE: Penalizes large errors, differentiable.

- **MAE:** Robust to outliers, simpler interpretation.